



Error handling

In programming, error handling is a complex topic with many pitfalls, and it can be even more daunting when you're writing code within the constraints of a framework, as the way you handle errors needs to mesh with the way the framework dispatches errors and vice versa.

This article paints the broad strokes of how errors are handled by the JavaScript framework and Owl, and gives some recommendations on how to interface with these systems in a way that avoids common problems.

Errors in JavaScript

Before we dive into how errors are handled in Odoo as well as how and where to customize error handling behavior, it's a good idea to make sure we're on the same page when it comes to what we mean exactly by "error", as well as some of the peculiarities of error handling in JavaScript.

The Error class

The first thing that may come to mind when we talk about error handling is the built-in `Error` class, or classes that extend it. In the rest of this article, when we refer to an object that is an instance of this class, we will use the term *Error object* in italics.

Anything can be thrown

In JavaScript, you can throw any value. It is customary to throw *Error objects*, but it is possible to throw any other object, and even primitives. While we don't recommend that you ever throw anything that is not an *Error object*, the Odoo JavaScript framework needs to be able to deal with these scenarios, which will help you understand some design decisions that we've had to make.

When instantiating an *Error object*, the browser collects information about the current state of the "call stack" (either a proper call stack, or a reconstructed call stack for async functions and promise continuations). This information is called a "stack trace" and is very useful for debugging. The Odoo framework displays this stack trace in error dialogs when available.

When throwing a value that is not an *Error object*, the browser still collects information about the current call stack, but this information is not available in JavaScript: it is only available in

allows us to filter errors based on their class when handling them. Unfortunately, JavaScript does not have syntactic support for filtering by error class in the catch clause, but you can relatively easily do it yourself:

```
try {
  doStuff();
} catch (e) {
  if (!(e instanceof MyErrorClass)) {
    throw e; // caught an error we can't handle, rethrow
  }
  // handle MyErrorClass
}
```

Promise rejections are errors

During the early days of Promise adoption, Promises were often treated as a way to store a disjoint union of a result and an “error”, and it was pretty common to use a Promise rejection as a way to signal a soft failure. While it might seem like a good idea at first glance, browsers and JavaScript runtimes have long started to treat rejected Promises the same way as thrown errors in pretty much every way:

- throwing in an async function has the same effect as returning a Promise that is rejected with the thrown value as its rejection reason.
- catch blocks in async functions catch rejected Promises that were awaited in the corresponding try block.
- runtimes collect stack information about rejected promises.
- a rejected Promise that is not caught synchronously dispatches an event on the global/window object, and if `preventDefault` is not called on the event, browsers log an error, and standalone runtimes like node kill the process.
- the debugger feature “pause on exceptions” pauses when Promises are rejected

For these reasons, the Odoo framework treats rejected Promises in the exact same way as thrown errors. Do not create rejected promises in places where you would not throw an error, and always reject Promises with *Error objects* as their rejection reason.

error events are not errors

With the exception of `error` events on the window, `error` events on other objects such as `<media>`, `<audio>`, `<img>`, `<script>` and `<link>` elements, or XMLHttpRequest objects are

```
const scriptEl = document.createElement("script");
scriptEl.src = "https://example.com/third_party_script.js";
return new Promise((resolve, reject) => {
  scriptEl.addEventListener("error", reject);
  scriptEl.addEventListener("load", resolve);
  document.head.append(scriptEl);
});
```

Lifecycle of errors within the Odoo JS framework

Thrown errors unwind their call stack to find a catch clause that can handle them. The way an error is handled depends on what code is encountered while unwinding the call stack. While there are a virtually infinite number of places errors could be thrown from, there are only a few possible paths into the JS framework's error handling code.

Throwing an error at the top-level of a module

When a JS module is loaded, the code at the top level of that module is executed and may throw. While the framework might report these errors with a dialog, module loading is a critical moment for the JavaScript framework and some modules throwing errors might prevent the framework code from booting entirely, so any error reporting at this stage is "best effort". Errors thrown during module loading should however always at the very least log an error message in the browser console. Because this type of error is critical, there is no way for the application to recover and you should write your code in such a way that it's impossible for the module to throw during definition. Any error handling and reporting that does happen at this stage is purely with the objective of helping you, the developer, fix the code that threw the error, and we provide no mechanism to customize how these errors are handled.

The error service

When an error is thrown but never caught, the runtime dispatches an event on the global object (`window`). The type of the event depends on whether the error was thrown synchronously or asynchronously: synchronously thrown errors dispatch an `error` event, and errors thrown from within an asynchronous context as well as rejected Promises dispatch an `unhandledrejection` event.

The JS framework contains a service that is dedicated to handling these events: the error service. When receiving one of these events, the error service starts by creating a new *Error*

value. The wrapping *Error object* is used to collect some information about the thrown value so that it can be used uniformly in the framework code that needs to display information about errors of any type.

The error service stores a complete stack trace of the thrown error on this wrapper *Error object* and, when the debug mode is `assets`, uses the source maps to add information in this stack trace about the source file that contains the function of each stack frame. The position of the function in the bundled assets is kept, as it can be useful in some scenarios. When errors have a `cause`, this process also unwinds the `cause` chain to build a complete composite stack trace. While the `cause` field on *Error objects* is standard, some major browsers still do not display the full stack trace of error chains. Because of this, we add this information manually. This is particularly useful when errors are thrown within Owl hooks, more on that later.

Once the wrapper error contains all the required information, we start the process of actually handling the error. To do this, the error service successively calls all functions registered in the `error_handlers` registry, until one of these functions returns a truthy value, which signals that the error has been handled. After this, if `preventDefault` was not called on the error event, and if the error service was able to add a stack trace on the wrapper error object, the error service calls `preventDefault` on the error event, and logs the stack trace in the console. This is because, as previously mentioned, some browsers do not display error chains correctly, and the default behaviour of the event is the browser logging the error, so we simply override that behaviour to log a more complete stack trace. If the error service was not able to collect stack trace information about the thrown error, we do not call `preventDefault`. This can happen when throwing non-error values: strings, undefined or other random objects. In those cases, the browser logs the stack trace itself, as it has that information but does not expose it to the JS code.

The `error_handlers` registry

The `error_handlers` registry is the main way to extend the way that the JS framework handles "generic" errors. Generic errors, in this context, means errors that can happen in many places, but that should be handled uniformly. Some examples:

- **UserError:** when the user attempts to perform an operation that the python code deems invalid for business reasons, the python code raises a `UserError`, and the `rpc` function throws a corresponding error in JavaScript. This has the potential to happen on any `rpc` anywhere, and we do not want developers to have to handle this kind of error explicitly in all those places, and we want the same behavior to happen everywhere: stop the currently

displayed the same way regardless of where it happens

- `LostConnection`: same reasoning again.

Throwing an error in an Owl component

Registering or modifying Owl components is the main way in which you can extend the functionality of the web client. As such, most errors that are thrown are in one way or another thrown from an Owl component. There are a few possible scenarios:

- Throwing in the component's setup or during rendering
- Throwing from within a lifecycle hook
- Throwing from an event handler

Throwing an error from an event handler or a function or method called directly or indirectly from an event handler means that neither Owl's code nor the JS framework's code is in the call stack. If you don't catch the error, it lands directly in the error service.

When throwing an error in a component's setup or during rendering, Owl catches the error and goes up the component hierarchy, allowing components that have registered error handlers with the `onError` hook to attempt to handle the error. If the error is not handled by any of them, Owl destroys the application as it is likely in a corrupted state.

See also

[Error handling in the Owl documentation](#) 

Inside Odoo, there are some places where we do not want the entire application to crash in case of error, and so the framework has a few places where it uses the `onError` hook. The action service wraps actions and views in a component that handles errors. If a client action or view throws an error during rendering, it attempts to go back to the previous action. The error is dispatched to the error service so that an error dialog can be shown regardless. A similar strategy is used in most places where the framework calls into "user" code: we generally stop displaying the faulty component and show an error dialog.

When throwing an error inside of a hook's callback function, Owl creates a new *Error object* that contains stack information about where the hook was registered, and sets its cause as the originally thrown value. This is because the stack trace of the original error contains no information about which component registered this hook and where, it only contains information about what called the hook. Because hooks are called by Owl code, most of this

make sure to read both parts of the composite stack trace:

```
Error: The following error occurred in onMounted: "My error"
  at wrapError
  at onMounted
  at MyComponent.setup
  at new ComponentNode
  at Root.template
  at MountFiber._render
  at MountFiber.render
  at ComponentNode.initiateRender

Caused by: Error: My error
  at ParentComponent.someMethod
  at MountFiber.complete
  at Scheduler.processFiber
  at Scheduler.processTasks
```

The first highlighted line tells you which component registered the `onMounted` hook, while the second highlighted line tells you which function threw the error. In this case, a child component is calling a function it received as prop from its parent, and that function is a method of the parent component. Both pieces of information can be useful, as the method could have been called by mistake by the child (or at a point in the lifecycle where it shouldn't), but it could also be that the parent's method contains a bug.

Marking errors as handled

In the previous sections, we talked about two ways to register error handlers: one is adding them to the `error_handlers` registry, the other is using the `onError` hook in owl. In both cases, the handler has to decide whether to mark the error as handled.

`onError`

In the case of a handler registered in Owl with `onError`, the error is considered by Owl as handled unless you rethrow it. Whatever you do in `onError`, the user interface is likely not synchronized with the state of the application, as the error prevented owl from completing some work. If you are unable to handle the error, you should rethrow it, and let the rest of the code handle it.

dispatch this error in a separate call stack outside of OWI. The easiest way to do this is to simply create a rejected Promise with the error as its rejection reason:

```
import { Component, onError } from "@odoo/owl";
class MyComponent extends Component {
  setup() {
    onError((error) => {
      // implementation of this method is left as an exercise for the reader
      this.removeErroringSubcomponent();
      Promise.reject(error); // create a rejected Promise without passing it anywhere
    });
  }
}
```

This causes the browser to dispatch an `unhandledrejection` event on the window, which causes the JS framework's error handling to kick in and deal with the error, in most cases by opening a dialog with information about the error. This is the strategy that is used internally by the action service and dialog service to stop rendering broken actions or dialogs while still reporting the error.

Handler in the `error_handlers` registry

Handlers that are added to the `error_handlers` registry can mark an error as being handled in two ways, with different meanings.

The first way is that the handler can return a truthy value, this means that the handler has processed the error and made something happen because the error it received matched the type of error it is able to handle. This generally means it has opened a dialog or notification to warn the user about the error. This prevents the error service from calling the following handlers with higher sequence number.

The other way is to call `preventDefault` on the error event: this has a different meaning. After deciding that it is able to handle the error, the handler needs to decide if the error it received is something that is allowed to happen during normal operation and if it is, it should call `preventDefault`. This is generally applicable to business errors such as an access errors or validation errors: users can share links with other users to resources to which they do not have access, and users can attempt to save a record that's in an invalid state.

Avoid throwing errors as much as possible

Errors introduce complexity in many ways, here are some reasons why you should avoid throwing them.

Errors are expensive

Because errors need to unwind the callstack and collect information as they do so, throwing errors is slow. Additionally, JavaScript runtimes are generally optimized with the assumption that exceptions are rare, and as such generally compile the code with the assumption that it doesn't throw, and fall back to a slower code path if it ever does.

Throwing errors makes debugging harder

JavaScript debuggers, like the one included in the Chrome and Firefox devtools for example, have a feature that allows you to pause the execution when an exception is thrown. You can also choose whether to pause only on caught exceptions, or on both caught and uncaught exceptions.

When you throw an error inside of code that is called by Owl or by the JavaScript framework (e.g. in a field, view, action, component, ...), because they manage resources, they need to catch errors and inspect them to decide whether the error is critical and the application should crash, or if the error is expected and should be handled in a particular manner.

Because of this, almost all errors that are thrown within JavaScript code are caught at some point, and although they may be rethrown if they cannot be handled, this means that using the "pause on uncaught exceptions" feature is effectively useless while working within Odoo, as it always pauses within the JavaScript framework code, instead of near the code that threw the error originally.

However, the "pause on caught exceptions" feature is still very useful, as it pauses execution on every throw statement and rejected promise. This allows the developer to stop and inspect the execution context whenever an exceptional situation occurs.

However, this is only true assuming that exceptions are rarely thrown. If exceptions are thrown routinely, any action within the page can cause the debugger to stop the execution, and the developer might need to step through many "routine" exceptions before they can get to the actual exceptional scenario they are interested in. In some situations, because clicking the play button in the debugger removes focus from the page, it may even make the interesting throw

Throwing breaks the normal flow of the code

When throwing an error, code that looks like it should always execute may be skipped, this can cause many subtle bugs and memory leaks. Here is a simple example:

```
eventTarget.addEventListener("event", handler);
someFunction();
eventTarget.removeEventListener("event", handler);
```

In this block of code, we add an event listener to an event target, then call a function which may dispatch events on that target. After the function call, we remove the event listener.

If `someFunction` throws, the event listener will never be removed. This means that the memory associated with this event listener is effectively leaked and will never be freed unless the `eventTarget` itself gets deallocated.

On top of the memory being leaked, the handler still being attached means that it may be called for events being dispatched for reasons other than the call to `someFunction`. This is a bug.

To account for this, one would need to wrap the call in a `try` block, and the cleanup in a `finally` block:

```
eventTarget.addEventListener("event", handler);
try {
    someFunction();
} finally {
    eventTarget.removeEventListener("event", handler);
}
```

While this now avoids the problems mentioned above, not only does this require more code, it also requires knowledge that the function may throw. It would be unmanageable to wrap all code that may throw in a `try/finally` block.

Catching errors

Sometimes, you need to call into code that is known to throw errors and you want to handle some of these errors. There are two important things to keep in mind:

you're trying to catch. Generally, the try block should contain exactly *one* statement.

```
let someVal;
try {
  someVal = someFunction();
  // do not start working with someVal here.
} catch (e) {
  if (!(e instanceof MyError)) {
    throw e;
  }
  someVal = null;
}
// start working with someVal here
```

While this is straightforward with try/catch, it's much easier to accidentally wrap a much larger portion of code in a catch clause when working with `Promise.catch` :

```
someFunction().then((someVal) => {
  // work with someVal
}).catch((e) => {
  if (!(e instanceof MyError)) {
    throw e;
  }
  return null;
});
```

In this example, the catch block is actually catching errors in the entire then block, which is not what we want. In this particular example, because we properly filter based on the error type, we're not swallowing the error, but you can see that it may be much easier to do so if we're expecting a single error type and decide not to have the instanceof check. Notice however that unlike the previous example, the null isn't going through the codepath that uses `someVal` . To avoid this, catch clauses should generally be as close as possible to the promise that may throw, and should always filter on the error type.

Error free control flow

example code:

```
let someVal;
try {
  someVal = someFunction();
} catch (e) {
  if (!(e instanceof MyError)) {
    throw e;
  }
  someVal = null;
}
```

There are many things that are problematic with this code. First, because we want the variable `someVal` to be accessible after the `try/catch` block, it needs to be declared before that block, and it cannot be `const` since it needs to be assigned after initialization. This hurts readability further down the road as you now have to look out for this variable potentially being reassigned later in the code.

Second, when we catch the error, we have to check that the error is actually the type of error we were expecting to catch, and if not, rethrow the error. If we don't do this, we might end up swallowing errors that were *actually* unexpected instead of reporting them correctly, e.g. we could be catching and swallowing a `TypeError` if the underlying code tries to access a property on `null` or `undefined`.

Lastly, not only is this very verbose, but it's easy to do this incorrectly: if you forget to add the `try/catch`, you are likely to end up with a traceback. If you add the `try/catch` block but forget to rethrow unexpected errors, you are swallowing unrelated errors. And if you want to avoid having to reassign the variable you may move the entire block that uses the variable inside the `try` block. The more code you have inside your `try` block, the more likely you are to catch unrelated errors, and swallow them if you forgot to filter by error type. It also adds an indentation level to the entire block, and you may even end up with nested `try/catch` blocks. Lastly, it makes it harder to identify which line is actually expected to throw the error.

The following sections outline some alternative approaches you can use instead of using errors.

Return `null` or `undefined`

```
const someVal = someFunction();  
// further  
if (someVal !== null) { /* do something */ }
```

As you can see, this is much simpler.

Return an object or array

In some cases, a value of `null` or `undefined` is part of the expected return values. In those cases, you can instead return a wrapper object or a two-element array that contains either the return value or the error:

```
const { val: someVal, err } = someFunction();  
if (err) {  
    return;  
}  
// do something with someVal as it is known to be valid
```

Or with an array:

```
const [err, someVal] = someFunction();  
if (err) {  
    return;  
}  
// do something with someVal as it is known to be valid
```

Note

When using a two-element array, it is advisable to have the error be the first element, so that it is harder to ignore by mistake when destructuring. One would need to explicitly add a placeholder or comma to skip the error, whereas if the error is the second element, it is easy to simply destructure only the first element and mistakenly forget to handle the error.

When to throw errors

everywhere; e.g., access errors can happen on basically any RPC, and we always want to display information about why the user doesn't have access.

- Some precondition that should always be fulfilled for some operation is not fulfilled; e.g., a view couldn't be rendered because the domain is invalid. These types of error are generally not intended to be caught anywhere and signal that code is incorrect or data is corrupted. Throwing forces the framework to bail out and prevents operating in a broken state.
- When traversing some deep data structure recursively, throwing an error can be more ergonomic and less error prone than having to manually test for errors and forward them through many levels of calls. This should be very rare in practice, and needs to be weighed against all the disadvantages mentioned in this article.

Get Help

[Contact Support](#)[Ask the Odoo Community](#)